ChatGPT

# Raising the Quality Bar for Generative AI Code in Production

## Introduction

Generative AI coding assistants are dramatically accelerating software development, enabling developers to produce code from brief prompts in a fraction of the time. Studies have found teams using AI-assisted coding tools can complete tasks up to 55% faster [1]. However, speed alone is not enough – especially when this AI-generated code is destined for production environments where failures are costly. Rushing thousands of lines of AI-written code into an enterprise system with a "just ship it" mentality is a recipe for hidden bugs, security vulnerabilities, and operational risks. In fact, treating AI-generated code as ready-to-deploy without extra scrutiny has led to subtle logic flaws and unvetted risks slipping past traditional reviews [2] [3].

To responsibly harness generative AI in software engineering, organizations must **raise the quality bar for AI-generated code going into production**. This white paper outlines a framework of **enhanced requirements and best practices** to ensure code produced by Large Language Models (LLMs) meets rigorous non-functional requirements (NFRs) before it ever touches a live system. The core idea is a shift in approach: instead of simply asking an AI to “write the code for feature X,” developers and tech leaders should require that **AI-assisted code contributions come with a full suite of supporting artifacts** – from design docs and threat models to test suites and runbooks. By making completeness and explainability mandatory, we can achieve the twin goals of **speed and reliability**. The following sections detail the challenges of scaling AI code in production, the new set of requirements proposed, the critical NFRs to enforce, and implementation strategies for tech teams and leadership.

## The Challenge of Scaling AI-Generated Code to Production

Many enterprises today rely on manual design and security reviews to catch issues in new code – a process that already struggles to scale. With the advent of GenAI-generated code, these traditional checkpoints become an even bigger bottleneck. AI can generate *vast amounts of code* rapidly, but reviewing thousands of lines by hand for every potential flaw is impractical. Furthermore, generative models often produce code that **“looks” correct but isn’t fully understood by the humans deploying it**. If teams naively trust AI outputs, they risk deploying code they cannot confidently maintain or secure. As one expert noted, large language models only mimic patterns from training data and have **“no real awareness of system behavior or risk boundaries”**, often omitting essential safeguards like input validation or permission checks [4]. Treating such code as inherently trustworthy leads to *shipping unvetted risk at scale*.

The current compliance model – where design documents and code changes go through slow security audits – does not scale to meet the velocity of AI-assisted development [2]. Yet simply bypassing those controls (“YOLO” deploying AI-written code) is not an option for production-critical systems. **We must therefore change our development requirements and workflows** to accommodate AI’s speed while maintaining (or improving) quality. In practice, this means shifting much more of the *responsibility for quality* onto the creation phase: developers using GenAI must ensure the output isn’t just functionally correct, but also meets all the non-functional criteria for production-readiness. The most successful

teams have already discovered this, implementing multi-layer review pipelines where automated tests, peer reviews for architecture, and security scans are mandatory before any AI-generated code is merged [1] [5]. In short, scaling GenAI in enterprise software demands a proactive, structured approach to **"trust but verify"**: leverage AI for speed, but insist on evidence and safeguards before trusting its code in live environments.

## Expanding the Requirements for AI-Generated Code

To make AI-assisted development viable for mission-critical systems, we propose an expanded set of deliverables for any code that will run in production. In the past, a developer (or an AI) might simply be given a feature request or bug fix and asked to deliver code changes. Going forward, the **developer or team using GenAI should be expected to deliver an entire package of artifacts** alongside the code. These artifacts cover design, testing, security, and documentation, and they ensure the new code can be understood, validated, and maintained. Specifically, for **any AI-generated code intended for production**, the following should be required:

- **Architecture and Design Brief** – A document (potentially AI-generated from the prompt) describing the proposed solution's design, high-level architecture, and rationale. This includes outlining how the new code fits into existing systems, what components or modules are affected, and the expected "blast radius" of the change. By having the AI or developer produce an explicit design brief, the team gains clarity on the change before diving into code. It also forces consideration of simpler designs. Indeed, providing structured requirements and architectural context to the AI tends to yield cleaner, more modular code [6] [7]. The design brief should highlight assumptions, constraints, and how key non-functional requirements (like performance or scalability) are addressed.

- **Threat Model and Risk Assessment** – A security review of the change, identifying potential vulnerabilities, abuse cases, or impact on compliance requirements. Rather than rely solely on a separate security team after the fact, the developer can prompt the AI to generate a preliminary threat model: e.g. listing entry points, data flows, and threat scenarios relevant to the new code. This document should enumerate risks (such as new external inputs the code will handle, data sensitivity, access control impacts, etc.) and how those risks are mitigated. Having an AI assist in writing the threat model can surface issues early, but human security experts should review it. In effect, **AI-generated code must be treated as *untrusted by default*** until proven otherwise [8]. Teams should introduce *stricter security controls and review gates* for GenAI outputs – for example, checking that the code doesn't unintentionally weaken authentication, introduce injection flaws, or misuse sensitive APIs [9] [10]. All high-risk changes might require a formal risk **approval** by security or compliance officers before deployment.

- **Readable, Well-Structured Code** – The code itself should meet a high standard of clarity and simplicity. This is perhaps the most important requirement: **no AI-generated code should go to production if the team cannot easily understand and explain its logic**. Developers should explicitly prompt the LLM to prioritize readability, concise logic, and adherence to the project's coding standards. The resulting code should avoid overly complex or clever constructs in favor of straightforward, maintainable approaches. Techniques like **encapsulating AI-generated code into small, well-defined functions or modules** are recommended to limit complexity [11]. Simpler code is not only easier to comprehend but also easier to test and less prone to hidden bugs. If the AI's first draft is convoluted, developers should iteratively refine the prompt or refactor the output until the code is clean. Remember, AI can confidently produce code that

*appears* correct but may hide subtle errors; insisting on simplicity and explicitness in code structure helps expose those errors to reviewers and tools.

- **Automated Tests (Unit, Integration, E2E)** – A comprehensive test suite must accompany the code. At a minimum, the contributor should provide unit tests that exercise the core logic, integration tests to ensure the new code works with existing components, and end-to-end tests (if applicable) covering the user-facing or system-level behavior. Generative AI can assist here as well – modern LLMs can generate unit test code given a function or module. These AI-written tests can then be reviewed and refined by developers. The goal is to ensure *confidence that the change works as intended and doesn't break existing functionality*. In fact, industry best practices now recommend that **each AI-generated code segment undergo automated testing immediately** to catch issues early [5] [12]. For example, one workflow is to generate code in small pieces and run tests against each piece right away, rather than generating a giant blob and testing at the end [12]. All tests should pass in the development environment before the code is even submitted for review. Furthermore, test coverage should meet or exceed the project's standard for human-written code [13] – if normally 80% line coverage is expected, AI code should hit the same bar or higher. This prevents the temptation to trust AI output without verification. **No AI-generated function should be merged without corresponding tests** proving it works in both expected and edge cases.

- **Validation & Deployment Checks** – Beyond regular tests, any production-bound code might require additional validation like performance testing, load testing, and deployment testing. For instance, if the change is to a critical service, the team should verify that performance meets requirements (through AI-assisted performance profiling or load tests) and that the code behaves under real-world conditions. AI may help generate scripts for these scenarios. Deployment tests (or **canary releases**) can be used as well – deploying the change to a small subset of users or a staging environment first. It's advisable never to promote AI-written code directly to 100% of production traffic without a phased rollout [14] [15]. Monitoring should be in place (see Observability below) during these canary deployments to detect any anomalies. The **infrastructure** for CI/CD should be configured to enforce these gates: e.g. blocking promotion if tests fail or if certain quality criteria are not met.

- **Runbook and Operational Documentation** – The introduction of new code should come with updates to **operational docs** such as runbooks, on-call rotation guides, or support FAQs. If the code introduces new alerts or metrics (as it ideally should for observability), those need to be documented for the SRE/Ops teams. The runbook should explain how to handle failures in the new component, any emergency remediation steps, feature flag toggles for the new code, etc. An AI can assist by generating a first draft of a runbook section describing the new functionality and its operation. The developer then curates this information to ensure accuracy. By updating runbooks concurrently with the code change, organizations avoid gaps where operators are blindsided by behavior they don't know how to manage.

- **Stakeholder Briefing and Debrief** – For significant features or changes, the developer should present the plan (brief) to relevant stakeholders (e.g. product managers, security leads, QA, operations) *before implementation*, and a debrief after implementation. In practice this might mean an AI-generated summary of what the change entails and its impact on various stakeholder concerns: user experience, security posture, compliance, etc. The *briefing* ensures everyone is on the same page about what will be delivered and any risks involved. The *debrief* (post-implementation review) allows the team to document what was done, what was learned, and confirm that all stakeholder concerns were addressed. Capturing these as short documents or meeting notes, aided by AI transcription/summarization, contributes to organizational

learning. It also creates an **audit trail** of accountability – showing that due diligence was done for the AI-generated code. In regulated industries, maintaining such audit trails of who reviewed what and which AI model was used can be crucial [16] [17].

- **User, Product, and Technical Documentation** – Finally, high-quality documentation for end users, product manuals, and technical reference must be produced or updated to reflect the new code. If a feature was added, the user guide or API docs should be revised accordingly. If internal APIs or modules changed, the internal developer docs need updating. AI can rapidly draft documentation from code or from developer prompts – for example, generating docstrings, README snippets, or even entire user manual sections from context [18] [19]. Documentation is not optional; it is a critical part of making the AI-generated code maintainable long-term. Thorough documentation provides transparency into how the AI was used and the rationale behind the code, which is invaluable for future debugging and knowledge transfer [20] [21]. In essence, **no code should enter production without the same level of documentation we would expect if a human wrote it (if not more)**, because other team members may be even more cautious about modifying AI-written sections unless they are clearly explained.

Requiring this complete set of deliverables might sound onerous, but many of these practices have always been considered *best-practice* in software engineering – they just were not consistently enforced due to time and resource constraints. Generative AI actually offers an opportunity to **improve our discipline** here. Since the AI can generate not only code but also tests, docs, and more, a single engineer can now produce all these artifacts with less effort than it would have taken a full team to do manually. For example, writing extensive documentation and unit tests by hand is often seen as tedious, but an AI assistant can draft them in seconds for the developer to refine [18]. The key is to **change the expectation**: if an AI coding tool is used, it should be used to its full potential by generating the *supporting materials* as well, not just the bare code. By setting this standard, organizations ensure that *every piece of AI-produced code comes with context and quality assurance baked in*. This comprehensive approach transforms AI code generation from a risky shortcut into a powerful accelerator that still upholds enterprise standards.

## Emphasizing Non-Functional Requirements (NFRs)

Central to this new approach is a relentless focus on **Non-Functional Requirements (NFRs)** – the qualities that make software robust, secure, and maintainable. Traditional functional requirements (the features and bug fixes) tell us *what* the code should do. NFRs, on the other hand, define *how* the code should perform and under what conditions, addressing aspects like security, reliability, maintainability, and performance. In the context of AI-generated code, NFRs must be treated as **first-class acceptance criteria**. AI systems left to their own devices will typically optimize for the immediate functional request (since the prompt usually describes a feature or fix) and might ignore aspects not explicitly asked for [22]. It's up to us to explicitly include NFRs in the AI's instructions and our review checklist. Below, we highlight the key NFRs that any production-bound generative AI code should fulfill:

- **Clarity & Maintainability:** The code must be easily understandable by human engineers. This means clear logic, consistent style, and adequate comments or explanations inline. A maintainable codebase is one where future developers (or even the original author, months later) can quickly comprehend what the code does and why. High maintainability reduces the chance of bugs when modifying the code later and lowers onboarding time for new team members. To achieve this with AI-generated code, one should instruct the AI to avoid obscure or overly terse constructs and perhaps even to include explanatory comments for complex sections. Research has shown that structured, requirement-driven prompts can nudge AI to

produce code with significantly better maintainability and readability [23] [24]. Teams can also measure this attribute using metrics like cyclomatic complexity or a maintainability index to ensure AI code isn't introducing excessive complexity [25]. The bottom line is **explainability**: if the code output is a black box to the team, it has no place in production. Code reviews should flag any AI-generated logic that cannot be easily traced or justified, and send it back for simplification or clarification.

- **Correctness & Reliability:** Beyond passing basic tests, the code should demonstrate reliability under real-world conditions. This involves not only functional correctness for expected inputs, but also robust handling of edge cases, error conditions, and unpredictable scenarios. AI-generated code has been known to fail in subtle ways outside the "happy path" – for instance, not handling null values, or giving poor error messages [26]. Thus, thorough testing (unit tests, integration tests, and beyond) is non-negotiable. Additionally, one should consider reliability features like graceful degradation (does the code fail safely if an external service is down?), retry logic for transient failures, and transactional integrity if applicable. Many of these aspects tie into how the code is designed (hence the importance of an architecture brief). By treating reliability as an NFR, teams using AI will, for example, explicitly prompt for input validation logic or error-handling paths that a naive AI solution might omit. All new code should meet the same reliability standards as hand-written code – if your system requires five-nines uptime, the AI component must not become the weak link. **Staged rollouts** and controlled deployments are a complementary practice here: exposing AI-written modules to increasing traffic only as they prove stable [14] [15].

- **Security:** Security is a critical non-functional requirement, given that AI-generated code can inadvertently introduce vulnerabilities. LLMs do not inherently understand secure coding practices; they might produce code that superficially works but fails to sanitize inputs, enforce authorization, or protect sensitive data. Common issues include using insecure defaults copied from examples (e.g. overly permissive access controls, hardcoded secrets, or missing encryption) [27]. As a result, **every AI-generated code change should undergo strict security scrutiny**. This begins with the earlier mentioned threat model artifact, but also includes automated scanning and review. Modern DevSecOps pipelines integrate static application security testing (SAST) and software composition analysis (for dependencies) as part of continuous integration – these should be run on AI-authored code as well, with no exemptions. In fact, experts advise that security scanning must be *mandatory, not optional* in AI coding workflows [28]. If the static analysis or dependency check flags a potential issue (e.g. an SQL injection vector or a vulnerable library call), the code should be fixed (potentially by re-prompting the AI with security requirements) before it goes forward. Additionally, dynamic testing (fuzz testing, penetration testing in a staging environment) can catch logical security flaws that static analysis might miss [9] [10]. The organization should also update its secure coding guidelines to include AI-specific considerations – for example, requiring that any use of generative code is accompanied by validation of its security properties, and treating AI contributions as **untrusted until verified** [8]. By holding AI code to the same (or higher) security standards as human code, companies can prevent the "unknown unknowns" – those silent flaws that only become apparent after a breach or incident.

- **Performance & Efficiency:** In production systems, performance is often a key concern – code must not only function correctly, but do so within acceptable time and resource limits. AI-generated code may not automatically optimize for performance; if not guided, an AI might produce a solution that uses significantly more memory or CPU than necessary, or that doesn't scale well with load. Non-functional requirements around performance (response time, throughput, resource consumption) should therefore be clearly communicated and tested. For

instance, if an AI is asked to generate a data processing function, the prompt or subsequent requirements should specify the expected data size and performance targets. After generation, performance testing or profiling should verify that the new code meets those targets. If not, the code might need refactoring (by a human or via an AI prompt focusing on optimization). In some cases, performance considerations might even affect architecture (e.g. using caching, concurrency, or a different algorithm), which is why having the AI produce an architecture draft helps catch inefficiencies early. While the voice memo did not explicitly highlight performance, it is an essential NFR for any production code. Including it in the criteria ensures that AI-accelerated development does not inadvertently degrade system efficiency or customer experience. Tools can assist here too: automated performance benchmarks can run in CI to compare the new code's efficiency against baseline. If there's a regression, it gets flagged before merge. In summary, GenAI code should be **as performant and scalable as if an expert human optimized it** – and if it isn't initially, the team must iterate to make it so, using AI assistance if helpful.

- **Observability & Operability:** Given the unpredictable nature of AI-generated logic, it is crucial to make such code transparent in operation. **Observability** is the ability to monitor and trace what the software is doing in production. Any code added via GenAI should include appropriate logging, metrics, and possibly distributed tracing so that engineers can understand its runtime behavior. For example, if an AI-generated module processes transactions, it should emit logs for key events (with relevant details, excluding sensitive data) and metrics such as number of transactions processed, error rates, processing time, etc. Observability was specifically highlighted as a requirement because it allows teams to detect anomalies in AI-written code early. Instrumenting the code with detailed logging and tagging those logs/metrics so one can distinguish AI-generated code paths is recommended [29]. By doing so, if an issue arises, it's easier to pinpoint if the AI-generated component is involved and diagnose the problem. This ties into reliability as well – you should set up **anomaly detection on telemetry from AI-generated features** to catch unexpected behavior [29] [30]. Operability extends beyond just metrics: it includes having feature flags or toggles for AI-introduced functionalities (so they can be turned off if they misbehave) and clear rollback procedures. Indeed, maintaining the ability to instantly disable or roll back an AI-deployed change is a wise safety net [31]. In essence, no AI-generated code should be a black box at runtime – it should be observable and controllable to the same extent as any well-engineered service.

- **Compliance & Auditability:** In enterprise contexts, especially regulated industries, compliance is a non-functional requirement that looms large. Deploying new code may require evidence that proper processes were followed (e.g. code review, testing, security sign-off) and that the code meets certain regulatory standards (for accessibility, data protection, etc.). AI-generated code should not be exempt from these obligations. Therefore, part of the deliverables (as noted) is comprehensive documentation not only of what the code does but **how it was produced and validated** [32]. For instance, keeping an audit log of the AI model used, the prompt given, and the human review comments can be extremely useful for compliance audits down the line [33]. If an issue ever arises from that code, the organization can trace back and see the chain of decisions. Some companies have begun tagging AI-authored commits in version control and tracking them separately [34] [35], which helps in compliance reporting and in measuring the impact of AI on development. Additionally, any domain-specific compliance (such as healthcare privacy rules, financial auditing requirements) should be explicitly included in the AI's requirements. For example, if generating code for a medical application, the prompt should remind the AI about HIPAA compliance for logging or data handling, as these are not things an AI would intuit on its own. By baking compliance checks into the process and ensuring

auditability, teams can safely integrate AI code into even the most sensitive production environments.

In summary, NFRs represent the quality guardrails for software, and they **must be front-and-center when using AI to generate code**. Many failures of AI-generated code can be traced back to an NFR being neglected – whether it's a security hole, a performance bottleneck, or unmaintainable logic. By contrast, when NFRs are clearly defined and enforced, AI can excel in helping meet them. It has been noted that explicitly stating NFRs in AI prompts significantly improves the compliance and effectiveness of the generated code [36] [37]. Tech leadership should update their definition of "done" for a task: it is not done when the code compiles or the feature works in a demo; it is only done when **all relevant quality attributes have been addressed** and verified. This mindset shift, more than any specific tool, will ensure that production software remains dependable even as we incorporate AI into our development workflows.

## Implementing a Robust GenAI Development Workflow

Adopting these enhanced requirements calls for changes in the development workflow and tooling. Organizations will need to evolve their practices and possibly build new infrastructure to support AI-augmented development. Below are key steps and considerations for integrating generative AI into the software lifecycle while maintaining high standards:

**1. Distinguish Experimentation from Production Work:** First and foremost, create a clear separation between *exploratory coding* with AI and *production-grade development* with AI. During prototyping or brainstorming, developers might freely use GenAI to spike out ideas or throwaway code without all the formalities – this is fine as long as that code stays out of mission-critical systems. However, once the decision is made that a piece of AI-assisted code will be part of a production deliverable, the process should switch to "production mode" with all the requirements discussed in this paper. Some companies even maintain separate environments or branches: one for AI experiments and another (tightly controlled) for integrating into mainline code [38]. This prevents experimental code from accidentally leaking into production without undergoing proper hardening.

**2. Develop Prompt Templates and Standards:** The way developers interact with the LLM is crucial. Teams should create standardized **prompt templates** that include sections for NFRs and documentation requests. For example, instead of a prompt that simply says *"Implement a REST API for updating a user profile,"* a structured prompt might say: *"Implement a REST API for updating a user profile in our system. Requirements: include input validation for all fields; enforce authorization such that only the profile owner or admin can update; ensure compliance with our data retention policy; produce code that is easy to understand and maintain. Additional output: also provide a brief documentation comment explaining the function's behavior, and suggest 3-5 unit tests."* This prompt explicitly feeds NFRs (security, maintainability) to the AI and asks for documentation and tests in the output. By standardizing such prompts (perhaps maintaining a library of them for common tasks), teams ensure consistency and reduce the chance of forgetting a critical requirement. As a bonus, having the AI generate tests and documentation in the same context can often tie them closely to the code's intent, catching misunderstandings early.

**3. AI Generation in Small Batches with Continuous Verification:** Encourage a workflow where AI-generated code is produced in **small, manageable chunks** rather than giant code dumps. After each chunk, perform immediate verification. For instance, a developer might use the AI to write one function or one module at a time, then run all relevant tests (unit tests for that function, integration tests if it touches external components) *immediately*. If tests fail or if static analysis flags issues, the developer

can correct the course (either by editing the code or re-prompting the AI with adjustments). This incremental approach is akin to the agile principle of iterative development, now applied to AI generation. It prevents a scenario where hundreds of AI-written functions are integrated only to discover a fundamental flaw that requires massive rework. As one guide on AI workflows puts it: *"Generate code in small, testable units rather than large modules. Immediately run automated tests against each generated segment"* [12]. This step-by-step validation creates a feedback loop with the AI: the results of tests and analyses can inform how the next prompt is shaped. Modern AI tooling can even incorporate test feedback to automatically refine outputs [39], but human oversight is the ultimate decision-maker at each step.

**4. Mandatory Code Reviews and Pair Programming with AI:** Human oversight is irreplaceable when it comes to understanding context and making judgment calls. All AI-generated code intended for production should undergo the same (or stricter) **peer code review** as human-written code. Organizations might update their review checklists to include specific items for AI-generated code, such as "Is the code easily understandable?", "Are there any suspicious or overly complex segments that need refactoring?", "Does the documentation/generated comments match the code's behavior?", and "Have all required tests and security scans been executed and passed?". Reviewers should approach AI-written code with healthy skepticism – assume nothing is obviously correct without proof. If something looks too clever or unfamiliar, they should question it, because it might be an artifact of the AI stitching patterns together. In addition, a great practice is **pair programming with AI**: treat the AI as a junior partner that can propose solutions, but have a senior developer guide the process. The senior developer can continuously review each function as it's generated, effectively doing a just-in-time code review. This aligns with the shift in developer role from coder to reviewer/architect that AI brings [40]. Remember that while AI speeds up writing code, the **validation and review remain the longest pole** – in fact, AI-assisted workflows often intentionally slow down the coding step to allow more time for reviewing logic and architecture [41] [42]. Embracing that trade-off is key to safe deployment.

**5. Integrate Quality Gates in the CI/CD Pipeline:** To enforce the new requirements, automate as much of the validation as possible in your continuous integration and deployment pipeline. For example, set up a pipeline that on each pull request runs: all the unit and integration tests; a static code analysis (for style guidelines and complexity); security scans (SAST, dependency checks); and perhaps even linting rules to catch inconsistent patterns. If any of these gates fail, the CI system should block the merge. Additionally, require that certain artifacts are present: one can use checklist bots or templates that force the author to include links or sections for the architecture doc, test evidence, etc., in the merge request description. Some teams have gone further to build **AI code review assistants** that annotate pull requests with potential issues or areas lacking clarity – these can be useful, but should complement, not replace, human code review [43] [44]. The goal is to make it nearly impossible to accidentally merge AI-generated code that hasn't been thoroughly vetted. It might also be wise to flag AI-generated contributions in version control (as mentioned earlier) [34], allowing specialized tracking. For instance, a policy could be: if a commit is tagged as AI-authored, it cannot be merged without at least two human approvals and all tests passing. By encoding policies in automation, we reduce reliance on memory or ad-hoc process – it becomes a systemic guardrail.

**6. Build Knowledge Repositories and Feedback Loops:** With more artifacts being produced (designs, threat models, docs, etc.), organizations should invest in a way to store and reuse this knowledge. A centralized repository for architecture decisions and threat models can prevent duplicated effort and help train both humans and AIs on the organization's standards. Over time, patterns will emerge – for example, the threat model for one microservice might be applicable to another, or a utility generated for one module can be reused. By collecting these outputs, you also create a **reference corpus** that could be used to further fine-tune AI models to your company's needs. If confidentiality allows, fine-tuning an AI on your internal codebase and documentation can greatly improve its relevance and

reduce hallucinations [45] [46]. Even without custom models, maintaining a library of good AI prompts and completions (as mentioned in step 2) allows continuous improvement. Teams should regularly review what AI did well and where it struggled, then feed those lessons into updated guidelines or prompt strategies. For instance, if an AI repeatedly makes a certain type of mistake (say, forgetting to handle time zone properly in date calculations), that insight can be added to a "best practices for AI prompts" document or lint rule. **Technical leadership** should treat the AI like a new developer in training – monitor its outputs, give it feedback, and gradually increase trust as it proves itself through consistent quality.

**7. Staged Deployment and Monitoring in Production:** After all the development-time checks are done, the first deployment of AI-generated code should be treated with extra care. Use *staged rollout* techniques – deploy to a limited subset of users or a beta environment first, and monitor closely [14] [15]. This was touched on earlier as part of NFRs (observability), but it's worth reiterating: production monitoring is the last line of defense to catch issues that all the pre-production testing might have missed. Track the behavior of the new code through logs and metrics. It is often helpful to compare metrics of the new code to the previous baseline if replacing existing functionality (for example, is error rate higher? Is latency within expected range?). If any anomaly is detected, automated alarms should notify engineers and ideally halt further rollout. Having feature flags around AI-generated features is extremely useful [31] – it enables quickly turning off the change without a new deployment if something goes wrong, buying time to investigate. Moreover, by gradually ramping up from (say) 5% to 50% to 100% of traffic over a couple of days, one can gain confidence at each step. This controlled exposure is a standard best practice for any deployment, but given the uncertainty with AI-created code, it's especially pertinent. Once the code proves itself in real usage, the organization can consider the experiment a success – and even then, continuous monitoring should remain in place, as part of normal operations.

Implementing the above workflow may require new tools and a mindset shift, but it directly addresses the scale and speed issues that naive AI coding would otherwise create. By **building a rigoro**us framework around generative coding, we allow AI to be used at scale *safely*. Tech leaders should champion these practices, ensuring their teams are trained in both using AI tools effectively and in the disciplined processes that must surround those tools.

## Benefits and Outcomes

Embracing this comprehensive approach to AI-generated code yields multiple benefits:

- **Higher Code Quality and Fewer Incidents:** With thorough testing, security analysis, and documentation in place for every change, the risk of production incidents drops significantly. Software developed with AI assistance can achieve defect rates on par with or even lower than traditional code when proper workflows are followed [47] [48]. Early adopters of robust AI development practices have reported no increase in production bugs, and in some cases faster incident resolution, because issues are easier to trace (thanks to good observability and documentation). The upfront effort in quality assurance prevents the far higher costs of downtime, security breaches, or emergency hotfixes later.

- **Scalability of Development:** By leveraging AI to generate not just code but also tests and docs, a small team can handle a larger scope of work without sacrificing standards. This effectively **scales up development capacity** safely. Organizations can take on more projects or deliver features faster, confident that each increment is solid. The approach also scales well with complex systems – since every change is accompanied by architectural insight and testing, even large codebases can grow in a controlled manner. The fear that "AI will create an unmaintainable

mess of code" is mitigated when each contribution is cleaned and vetted. In fact, teams find they can accelerate feature delivery (some report cycle time reductions of 35–55%) once the initial learning curve of the AI workflow is overcome [47] [49], all while keeping quality constant.

- **Better Knowledge Retention and Collaboration:** The requirement for explicit documentation, design briefs, and debriefs means that less knowledge remains "tacit" or only in a developer's head. Over time, this leads to a rich knowledge base about the system. New hires or cross-functional team members can get up to speed faster by reading the accumulated architecture briefs and decision logs that accompany the code. In an AI-assisted environment, where code may be written by a non-human agent, having this context captured is even more crucial for trust. It also fosters collaboration – security, ops, and product stakeholders are looped in through briefs and risk assessments, breaking down silos. Everyone speaks a common language of explicit requirements and verified outcomes, rather than vague assurances. This can uplift the overall engineering maturity of the organization.

- **Increased Developer Productivity & Satisfaction:** While it might seem that all these extra steps add burden, developers often appreciate the clarity and support. The AI handles the grunt work of drafting tests or documentation, letting engineers focus on higher-level problem solving and fine-tuning. Developers spend more time on creative design and critical thinking (reviewing, refining, risk assessing) and less on rote coding, which many find a more rewarding use of their skills [40] [41]. Furthermore, knowing that the code they ship is well-tested and documented gives developers confidence and reduces stress when pushing to production. Some surveys have indicated that developers feel **more confident in code quality** with AI assistance when proper checks are in place, as the AI can catch issues they might overlook and vice versa [50] [51]. Over time, this can improve team morale and openness to adopting AI tools, as the team sees them as beneficial collaborators rather than risky shortcuts.

- **Establishing Trust in AI within the Organization:** Finally, by demonstrating that AI-generated code can be as robust as human code, tech leadership builds trust with executive stakeholders and customers regarding AI in the software development process. This proactive stance on governance and quality turns AI into a competitive advantage rather than a liability. It sends a message that the company is *responsibly innovative* – leveraging cutting-edge tools while upholding the highest standards of engineering. This can be important for public perception, regulatory comfort, and the company's brand. Conversely, organizations that skip these practices may face highly visible failures or security incidents that erode trust in AI solutions. Thus, investing in rigorous AI development practices is also an investment in the company's reputation and future readiness.

## Conclusion

Generative AI is poised to transform software development, but without the right practices, it can just as easily generate chaos. To safely unlock AI's potential, organizations must **evolve their development lifecycles** to demand more than just working code – they must demand *understood*, *tested*, *secure*, and *well-documented* code. In this white paper, we advocated for a comprehensive set of requirements for AI-generated code destined for production: including design documentation, threat models, complete test coverage, observability, and more. We emphasized treating non-functional requirements as equal in importance to functional correctness, and leveraging AI itself to help fulfill those quality needs.

By holding AI-assisted development to a higher standard, enterprises will actually find themselves able to move faster **with confidence**. The initial voice that inspired this paper put it plainly: *"the more you*

*know, this is what allows you to go faster."* By ensuring each change is understandable and monitored, we reduce the fear of change that slows many organizations down. In practical terms, that means no code enters production that engineers can't explain and trust. It means building the infrastructure and culture such that AI-generated outputs are thoroughly vetted and enhanced before they ever impact real users.

The path forward involves investment in tooling, training, and perhaps most importantly, a mindset shift. Developers must become comfortable acting as reviewers and architects for AI-produced code, and leaders must insist on process discipline around AI usage. The payoff is software that can evolve at the speed of AI, while *operating at the reliability of well-engineered human code*. Firms that achieve this balance will not only prevent the pitfalls of careless AI coding but will outpace competitors through superior development agility.

In closing, generative AI can be a powerful ally in delivering quality software at scale – but only if we raise our expectations and require it to deliver *the whole package*, not just fragmented solutions. The era of "YOLO" deployments of black-box AI code should be left behind. By implementing the practices outlined here and focusing on robust non-functional requirements, organizations can confidently integrate AI into their production pipelines. The result is a new paradigm of development: one where humans and AI collaborate to produce software faster than ever, and with assurance that every line of code is production-ready. The future of software belongs to those who can innovate swiftly **and** safely – with generative AI and strong engineering governance hand-in-hand, we can achieve both.

**Sources:**


1. Aimensa, *"AI Coding Workflow Best Practices for Production-Ready Code"* [1] [5] – Explains that AI-generated code should be treated as a first draft, with rigorous testing (unit, integration, performance) and review gates to ensure production quality. Also notes the shift in developer focus to validation and architecture when using AI [40].

2. Codacy Blog, *"Best Practices for Coding with AI in 2024"* [52] [53] – Emphasizes thorough review and testing of AI-generated code. Advises developers to understand and adapt AI outputs, use static analysis, enforce code reviews for all auto-generated code, and maintain high test coverage before releasing AI-written code.

3. Inflectra, *"Why Your AI-Generated Code Needs Structured Requirements"* [36] [37] – Highlights the importance of explicitly stating non-functional requirements (security, maintainability, performance, etc.) when using AI tools. Notes that ambiguous or absent NFRs lead to poor outcomes, while clearly articulated NFRs significantly improve AI-generated code's quality and compliance.

4. AppSecEngineer, *"AI-Generated Code Needs Its Own Secure Coding Guidelines"* [4] [9] – Warns that AI models produce code without understanding system context or threats, often omitting critical security measures. Recommends treating AI code as untrusted and implementing targeted security reviews and tests (privilege scope, data handling, auth flows) specifically for AI outputs to catch design flaws that traditional checks might miss.

5. Aimensa, *"AI Coding Workflow – Deployment and Observability"* [29] [32] – Advises never deploying AI code directly to production, instead using staged rollouts and strong observability. Recommends instrumentation (logging, metrics, tracing) for all AI-generated code and

maintaining audit trails (documenting which model/prompt was used, test results, and review notes) to ensure maintainability and compliance in enterprise environments.

---


1 5 12 13 14 15 16 17 28 29 30 31 32 33 34 35 38 39 40 41 42 45 46 47 48 49 AI Coding Workflow: Production-Ready Code [Guide]| Aimensa
https://aimensa.com/ai-coding-workflow-production-practices

2 3 4 8 9 10 27 AI-Generated Code Needs Its Own Secure Coding Guidelines
https://www.appsecengineer.com/blog/ai-generated-code-needs-its-own-secure-coding-guidelines

6 7 23 24 25 36 37 Why Your AI-Generated Code Needs Better Requirements | Infle
https://www.inflectra.com/Ideas/Topic/Why-Your-AI-Generated-Code-Needs-Better-Requirements.aspx

11 18 19 20 21 43 44 50 51 52 53 Best Practices for Coding with AI in 2024
https://blog.codacy.com/best-practices-for-coding-with-ai

22 Using Copilot to Generate Code? Key Issues to Watch For - Substack
https://substack.com/home/post/p-182678304

26 Hidden Challenges of Testing AI-Generated Code (And How to ...
https://qualizeal.com/hidden-challenges-of-testing-ai-generated-code/